Computational Physics Hw_4

Student ID: 2024141230044

Name: Fanyi

Problem 2

For the function

$$f(x) = (x-1)^{10},$$

compare three evaluation methods:

1. direct evaluation of $(x-1)^{10}$,
2. direct evaluation of the expanded polynomial,
3. Horner's method for the expanded polynomial.

The calculations are carried out in single precision (float), double precision (double), and quad precision (`__float128`). The interval is zoomed to $x \in [0.7, 1.3]$, and the relative error distribution of the expansion is examined. Finally, Horner's method is compared with the naive expansion and the numerical behavior is explained.

目录

1 Algorithm Principle **3**

1.1 Three equivalent formulas 3

1.2 Reference value and relative error 3

2 Program Code **4**

3 Numerical Results **6**

3.1 Zoomed function plots 6

3.2 Relative error distribution 8

3.3 Representative data points 9

4 Discussion **10**

4.1 Why does the expansion fail near $x = 1$? 10

4.2 Why is Horner's method better? 10

4.3 What is the best formula? 11

5 Conclusion **11**

1 Algorithm Principle

1.1 Three equivalent formulas

For the same mathematical function,

$$f(x) = (x-1)^{10},$$

we compare the following three implementations.

Direct form. Compute

$$f(x) = (x-1)^{10}$$

by first forming $x-1$, then multiplying ten times.

Expanded polynomial. Use the binomial expansion

$$\begin{aligned}(x-1)^{10} = x^{10} - 10x^9 + 45x^8 - 120x^7 + 210x^6 \\ - 252x^5 + 210x^4 - 120x^3 + 45x^2 - 10x + 1.\end{aligned}$$

This form is algebraically correct, but near $x = 1$ the individual terms are large and nearly cancel, so it is sensitive to roundoff.

Horner form. Rewrite the same polynomial as

$$\begin{aligned}f(x) =&((((((((((x-10)x+45)x-120)x+210)x-252) \\ &x+210)x-120)x+45)x-10)x+1.\end{aligned}$$

Horner's method reduces the number of multiplications and avoids explicitly computing high powers. It is usually more stable than the naive expanded form, although it still evaluates the same cancellation-prone polynomial.

1.2 Reference value and relative error

The program uses quad precision direct evaluation as the reference value:

$$f_{\text{ref}}(x) = \texttt{direct_q}(x).$$

For an approximation $f_{\text{approx}}(x)$, the relative error is

$$\text{relerr}(x) = \left| \frac{f_{\text{approx}}(x) - f_{\text{ref}}(x)}{f_{\text{ref}}(x)} \right|.$$

At $x = 1$, the exact value is 0, so the relative error is undefined; the program writes nan at that point.

2 Program Code

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <quadmath.h>

typedef __float128 quad_t;

static const int C[11] = {1, -10, 45, -120, 210, -252, 210, -120,
   45, -10, 1};

static float direct_f(float x) {
   float t = x - 1.0f;
   float y = 1.0f;
   for (int i = 0; i < 10; ++i) y *= t;
   return y;
}

static double direct_d(double x) {
   double t = x - 1.0;
   double y = 1.0;
   for (int i = 0; i < 10; ++i) y *= t;
   return y;
}

static quad_t direct_q(quad_t x) {
   quad_t t = x - (quad_t)1.0;
   quad_t y = (quad_t)1.0;
   for (int i = 0; i < 10; ++i) y *= t;
   return y;
}
```

```
static float expand_f(float x) {
   float p[11];
   p[0] = 1.0f;
   for (int i = 1; i <= 10; ++i) p[i] = p[i - 1] * x;
   float y = 0.0f;
   for (int i = 0; i <= 10; ++i) y += (float)C[i] * p[10 - i];
   return y;
}

static double expand_d(double x) {
   double p[11];
   p[0] = 1.0;
   for (int i = 1; i <= 10; ++i) p[i] = p[i - 1] * x;
   double y = 0.0;
   for (int i = 0; i <= 10; ++i) y += (double)C[i] * p[10 - i];
   return y;
}

static quad_t expand_q(quad_t x) {
   quad_t p[11];
   p[0] = (quad_t)1.0;
   for (int i = 1; i <= 10; ++i) p[i] = p[i - 1] * x;
   quad_t y = (quad_t)0.0;
   for (int i = 0; i <= 10; ++i) y += (quad_t)C[i] * p[10 - i];
   return y;
}

static float horner_f(float x) {
   float y = (float)C[0];
   for (int i = 1; i <= 10; ++i) y = y * x + (float)C[i];
   return y;
}

static double horner_d(double x) {
   double y = (double)C[0];
   for (int i = 1; i <= 10; ++i) y = y * x + (double)C[i];
   return y;
}

static quad_t horner_q(quad_t x) {
   quad_t y = (quad_t)C[0];
```

```
    for (int i = 1; i <= 10; ++i) y = y * x + (quad_t)C[i];
    return y;
}
```

3 Numerical Results

The program samples 6001 equally spaced points on $[0.7, 1.3]$. It outputs two CSV files:

- `values.csv`: the computed function values for all methods;
- `relerr.csv`: the relative errors against quad direct evaluation.

3.1 Zoomed function plots

Figures 1–3 compare the three formulas in float, double, and quad precision.

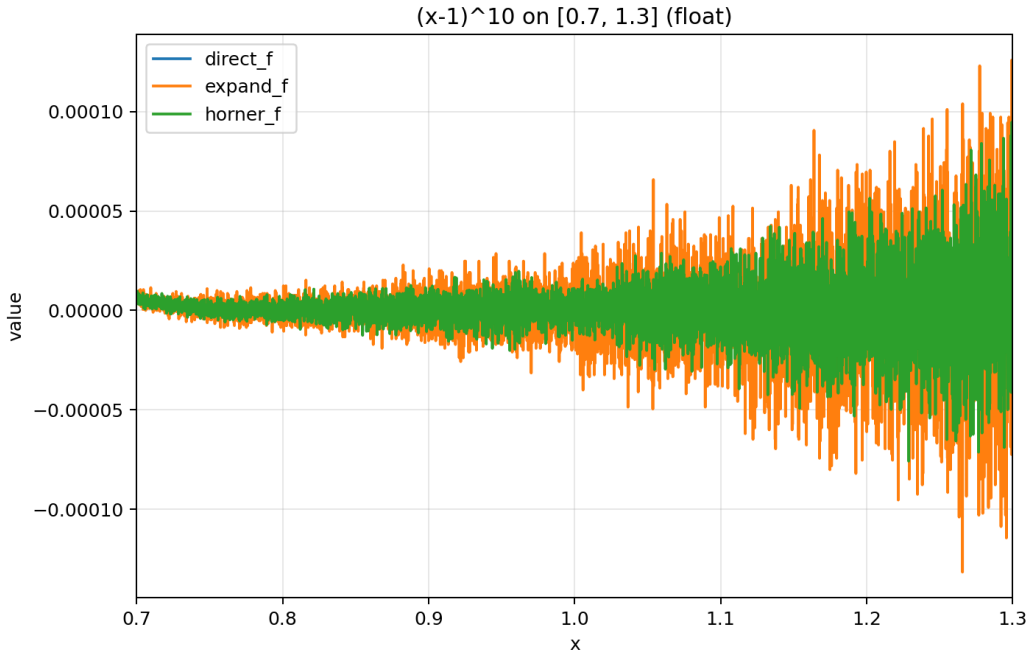

图 1: Function values on $[0.7, 1.3]$ in float precision.

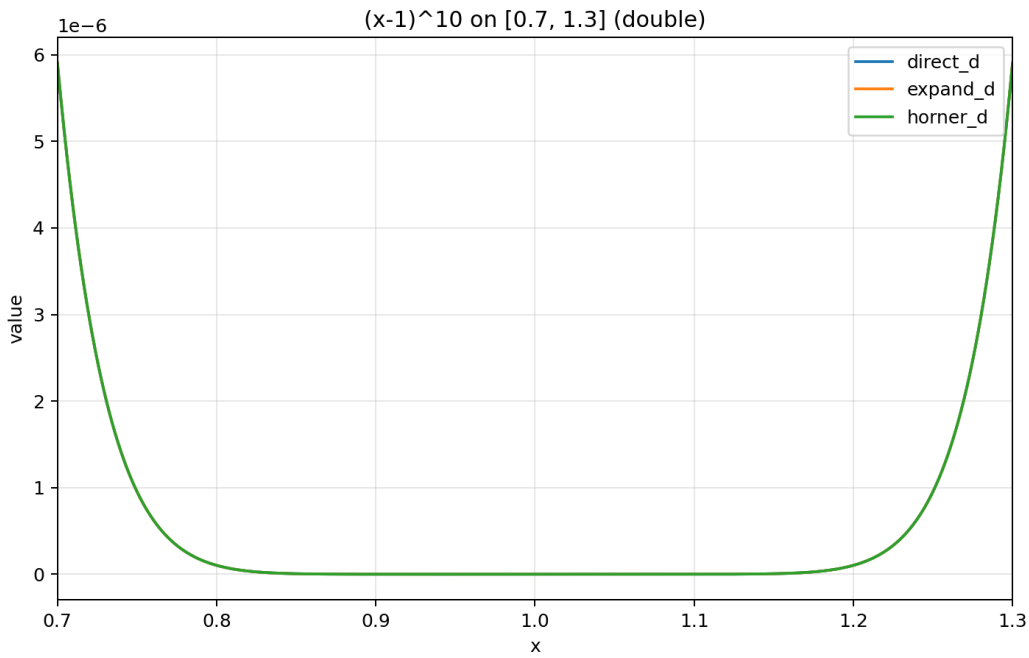

图 2: Function values on $[0.7, 1.3]$ in double precision.

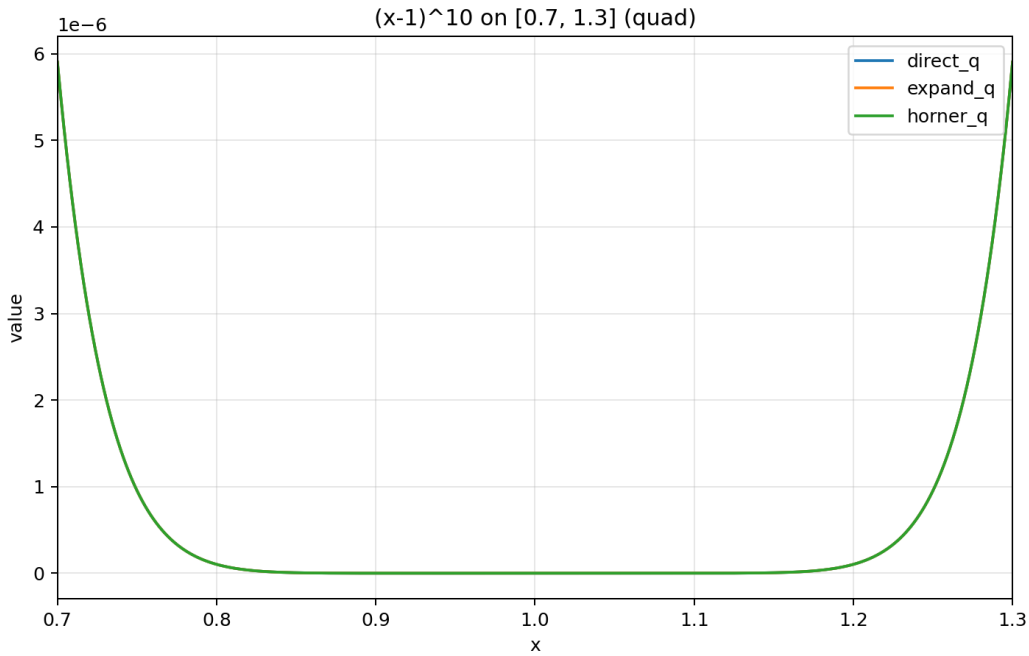

图 3: Function values on $[0.7, 1.3]$ in quad precision.

From the plots, the direct evaluation remains smooth and consistent across precisions. By contrast, the naive expanded form oscillates strongly near $x = 1$, especially in float and double precision. Horner's method improves the expanded evaluation, but it still shows visible deviation near $x = 1$ because the underlying cancellation problem is still present.

3.2 Relative error distribution

Figures 4–6 show the relative-error distributions. Since the exact value becomes extremely small near $x = 1$, even tiny absolute perturbations can produce enormous relative errors.

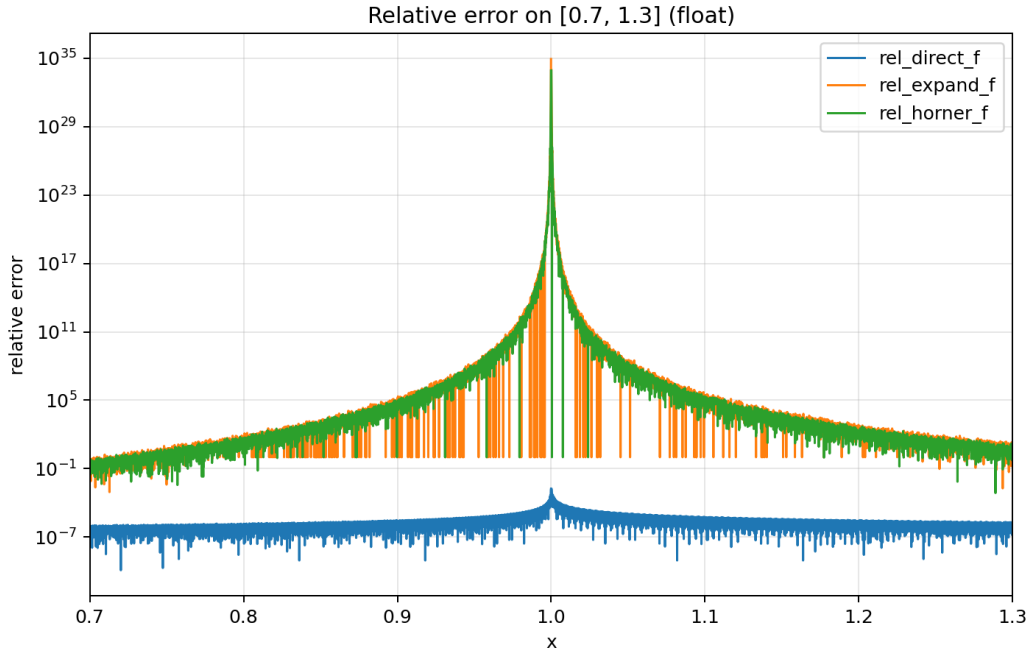

图 4: Relative error in float precision.

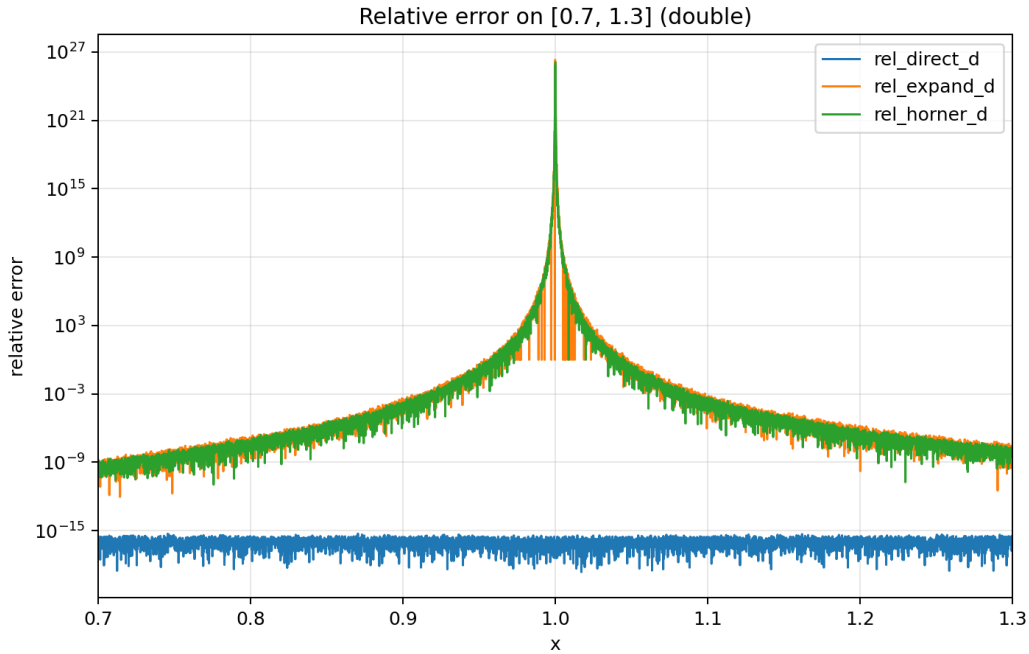

图 5: Relative error in double precision.

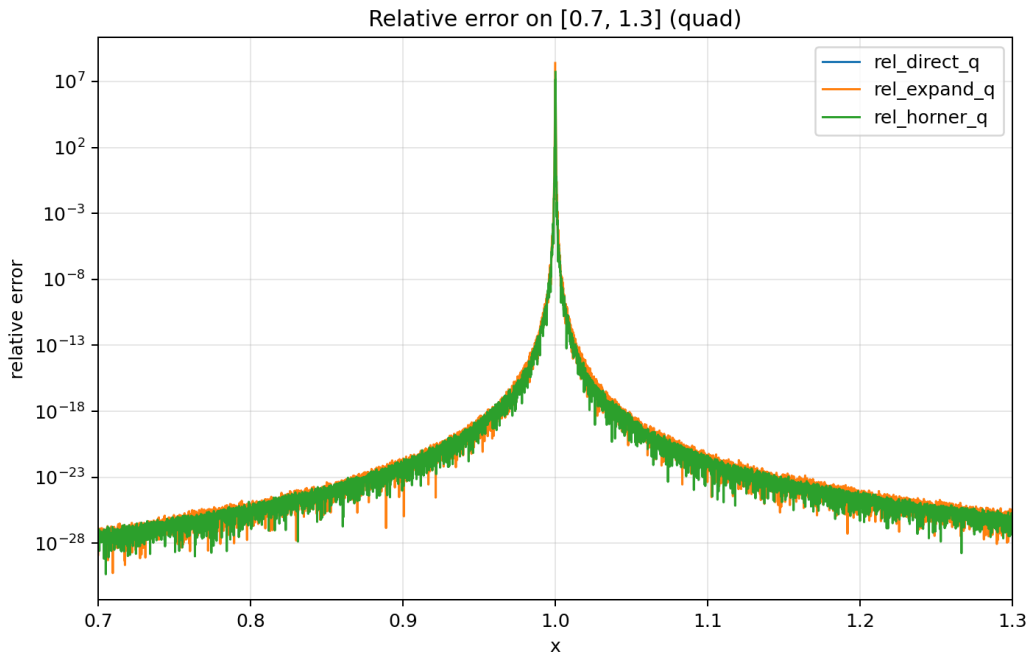

图 6: Relative error in quad precision.

3.3 Representative data points

A few representative points illustrate the behavior more clearly.

Table 1: Representative double-precision results.

x	direct_d	expand_d	horner_d
0.90	1.000000×10^{-10}	1.000160×10^{-10}	1.000179×10^{-10}
0.99	1.000000×10^{-20}	1.953993×10^{-14}	3.219647×10^{-15}
0.9999	1.000000×10^{-40}	$-1.953993 \times 10^{-14}$	$-5.995204 \times 10^{-15}$
1.0001	1.000000×10^{-40}	1.243450×10^{-14}	$-1.154632 \times 10^{-14}$
1.01	1.000000×10^{-20}	5.151435×10^{-14}	$-8.881784 \times 10^{-16}$
1.10	1.000000×10^{-10}	1.000089×10^{-10}	1.000238×10^{-10}

Table 2: Representative relative errors in double precision.

x	relerr(direct_d)	relerr(expand_d)	relerr(horner_d)
0.90	6.96×10^{-17}	1.60×10^{-4}	1.79×10^{-4}
0.99	2.42×10^{-16}	1.95×10^{6}	3.22×10^{5}
0.9999	7.85×10^{-17}	1.95×10^{26}	5.99×10^{25}
1.0001	7.85×10^{-17}	1.24×10^{26}	1.15×10^{26}
1.01	2.42×10^{-16}	5.15×10^{6}	8.88×10^{4}
1.10	5.66×10^{-17}	8.89×10^{-5}	2.38×10^{-4}

These tables show three main facts:

1. The direct form is the most reliable, because it works with the small quantity $x - 1$ from the start.
2. The expanded polynomial is extremely unstable near $x = 1$, where large terms cancel to produce a tiny exact value.
3. Horner's method is clearly better than naive expansion, but it does not completely remove the cancellation built into the expanded polynomial.

4 Discussion

4.1 Why does the expansion fail near $x = 1$?

When $x \approx 1$, each term in the expanded polynomial,

$$x^{10},\ -10x^9,\ 45x^8,\ \ldots,\ -10x,\ 1,$$

is individually of order 1, but their final sum is of order $|x-1|^{10}$, which can be as small as 10^{-40} for $x = 1 \pm 10^{-4}$. Thus the result is obtained by subtracting several nearly equal numbers, and the final tiny value is easily buried by roundoff. This is a 典型的 catastrophic cancellation example.

4.2 Why is Horner's method better?

Horner's method evaluates the polynomial with fewer arithmetic operations and without forming every power separately. Therefore it reduces the accumulation of roundoff and repeated recomputation. This is why Horner's curve is usually closer to the direct result than the naive expanded curve. However,

Horner's method still evaluates the expanded polynomial itself, so it cannot fully avoid the loss of significance near $x = 1$.

4.3 What is the best formula?

For this problem, the best practical formula is the direct one:

$$(x - 1)^{10}.$$

It aligns the computation with the true scale of the answer. The closer the algebraic form is to the numerical structure of the result, the more stable the computation tends to be.

5 Conclusion

The numerical experiment satisfies all requested tasks for Problem 2:

- the function was computed in single, double, and quad precision;
- the plots were zoomed to $x \in [0.7, 1.3]$;
- the relative error distribution was drawn;
- Horner's method was implemented and compared with the direct expansion.

The conclusion is clear: near $x = 1$, the expanded polynomial suffers severe cancellation, producing extremely large relative errors. Horner's method alleviates the problem but does not eliminate it. Direct evaluation of $(x - 1)^{10}$ is the most numerically stable method.